

第十二次上机作业

学号：241830137 姓名：朱子健

2026 年 6 月 17 日

1 问题描述

考虑如下常微分方程初值问题：

$$\begin{cases} y' = -\frac{1}{x^2} - \frac{y}{x} - y^2, & 1 \leq x \leq 2, \\ y(1) = -1. \end{cases} \quad (1)$$

分别用 Euler 方法、改进的 Euler 方法、二阶 Heun 方法、中点方法和经典四阶 Runge-Kutta 方法求解上述初值问题，列表、画图比较他们的计算结果。

2 算法原理

考虑一阶常微分方程初值问题

$$y' = f(x, y), \quad y(x_0) = y_0.$$

取步长 $h = (b - a)/N$ ，节点 $x_n = x_0 + nh$ ，数值解记为 $y_n \approx y(x_n)$ 。

1. Euler 方法（一阶）

格式：

$$y_{n+1} = y_n + hf(x_n, y_n).$$

算法 1 Euler 方法一步推进

输入：当前点 (x_n, y_n) ，步长 h

输出：下一节点数值解 y_{n+1}

1: 计算右端函数值 $f_n \leftarrow f(x_n, y_n)$

2: $y_{n+1} \leftarrow y_n + h \cdot f_n$

3: **return** y_{n+1}

2. 改进的 Euler 方法（二阶显式梯形法）

格式：

$$\begin{aligned} \tilde{y}_{n+1} &= y_n + hf(x_n, y_n), \\ y_{n+1} &= y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, \tilde{y}_{n+1})]. \end{aligned}$$

算法 2 改进的 Euler 方法一步推进

输入: 当前点 (x_n, y_n) , 步长 h

输出: 下一节点数值解 y_{n+1}

- 1: $f_n \leftarrow f(x_n, y_n)$
 - 2: $\tilde{y}_{n+1} \leftarrow y_n + h \cdot f_n$ {预测步}
 - 3: $\tilde{f}_{n+1} \leftarrow f(x_n + h, \tilde{y}_{n+1})$
 - 4: $y_{n+1} \leftarrow y_n + \frac{h}{2}(f_n + \tilde{f}_{n+1})$ {校正步}
 - 5: **return** y_{n+1}
-

3. 二阶 Heun 方法

根据讲义式 (5.3.9), 取参数 $a_2 = \frac{2}{3}$, 得

$$K_1 = f(x_n, y_n), \quad K_2 = f\left(x_n + \frac{2}{3}h, y_n + \frac{2}{3}hK_1\right),$$

$$y_{n+1} = y_n + \frac{h}{4}(K_1 + 3K_2).$$

算法 3 二阶 Heun 方法一步推进

输入: 当前点 (x_n, y_n) , 步长 h

输出: 下一节点数值解 y_{n+1}

- 1: $K_1 \leftarrow f(x_n, y_n)$
 - 2: $K_2 \leftarrow f\left(x_n + \frac{2}{3}h, y_n + \frac{2}{3}hK_1\right)$
 - 3: $y_{n+1} \leftarrow y_n + \frac{h}{4}(K_1 + 3K_2)$
 - 4: **return** y_{n+1}
-

4. 中点方法 (变形的 Euler 方法)

格式:

$$y_{n+1} = y_n + hf\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}f(x_n, y_n)\right).$$

算法 4 中点方法一步推进

输入: 当前点 (x_n, y_n) , 步长 h

输出: 下一节点数值解 y_{n+1}

- 1: $K_1 \leftarrow f(x_n, y_n)$
 - 2: $y_{n+1} \leftarrow y_n + h \cdot f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1\right)$
 - 3: **return** y_{n+1}
-

5. 经典四阶 Runge-Kutta 方法

格式:

$$\begin{aligned}K_1 &= f(x_n, y_n), \\K_2 &= f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1\right), \\K_3 &= f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_2\right), \\K_4 &= f(x_n + h, y_n + hK_3), \\y_{n+1} &= y_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4).\end{aligned}$$

算法 5 经典四阶 Runge-Kutta 方法一步推进

输入: 当前点 (x_n, y_n) , 步长 h

输出: 下一节点数值解 y_{n+1}

```
1:  $K_1 \leftarrow f(x_n, y_n)$ 
2:  $K_2 \leftarrow f(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1)$ 
3:  $K_3 \leftarrow f(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_2)$ 
4:  $K_4 \leftarrow f(x_n + h, y_n + hK_3)$ 
5:  $y_{n+1} \leftarrow y_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4)$ 
6: return  $y_{n+1}$ 
```

3 精确解

原方程可化为 $u = xy$ 的方程

$$u' = -\frac{1+u^2}{x},$$

分离变量并利用初值 $y(1) = -1$ (即 $u(1) = -1$) 得

$$u(x) = -\tan\left(\ln x + \frac{\pi}{4}\right),$$

故精确解为

$$y(x) = -\frac{1}{x} \tan\left(\ln x + \frac{\pi}{4}\right).$$

4 数值实验与结果分析

取步长 $h = 0.1$, 即 $N = 10$, 计算节点 $x_i = 1.0, 1.1, \dots, 2.0$ 处的数值解。表 1 列出了五种方法的计算结果及其与精确解的绝对误差, 图 1 给出了相应的曲线对比。

表 1: 五种方法数值结果与误差比较 ($h = 0.1$)

x	节点 精确解	Euler		改进 Euler		Heun		中点		RK4	
		数值解	误差	数值解	误差	数值解	误差	数值解	误差	数值解	误差
1.0	-1.000000	-1.000000	0.0e+00	-1.000000	0.0e+00	-1.000000	0.0e+00	-1.000000	0.0e+00	-1.000000	0.0e+00
1.1	-0.995897	-0.995463	4.3e-04	-0.995904	7.0e-06	-0.995896	5.5e-07	-0.995893	3.4e-06	-0.995897	7.9e-08
1.2	-0.987734	-0.987070	6.6e-04	-0.987746	1.2e-05	-0.987734	2.0e-07	-0.987731	3.6e-06	-0.987734	1.7e-08
1.3	-0.977581	-0.976731	8.5e-04	-0.977598	1.7e-05	-0.977580	4.0e-07	-0.977577	3.9e-06	-0.977581	2.7e-08
1.4	-0.966342	-0.965333	1.0e-03	-0.966364	2.2e-05	-0.966341	6.2e-07	-0.966338	4.2e-06	-0.966342	3.7e-08
1.5	-0.954488	-0.953343	1.1e-03	-0.954514	2.6e-05	-0.954487	8.5e-07	-0.954483	4.6e-06	-0.954488	4.7e-08
1.6	-0.942279	-0.941018	1.3e-03	-0.942309	3.0e-05	-0.942277	1.1e-06	-0.942273	5.0e-06	-0.942279	5.8e-08
1.7	-0.929847	-0.928486	1.4e-03	-0.929880	3.3e-05	-0.929845	1.3e-06	-0.929840	5.4e-06	-0.929847	6.8e-08
1.8	-0.917252	-0.915805	1.4e-03	-0.917288	3.6e-05	-0.917249	1.6e-06	-0.917244	5.8e-06	-0.917252	7.8e-08
1.9	-0.904526	-0.903005	1.5e-03	-0.904564	3.8e-05	-0.904523	1.8e-06	-0.904518	6.2e-06	-0.904526	8.9e-08
2.0	-0.891688	-0.890102	1.6e-03	-0.891729	4.0e-05	-0.891684	2.1e-06	-0.891679	6.6e-06	-0.891688	9.9e-08

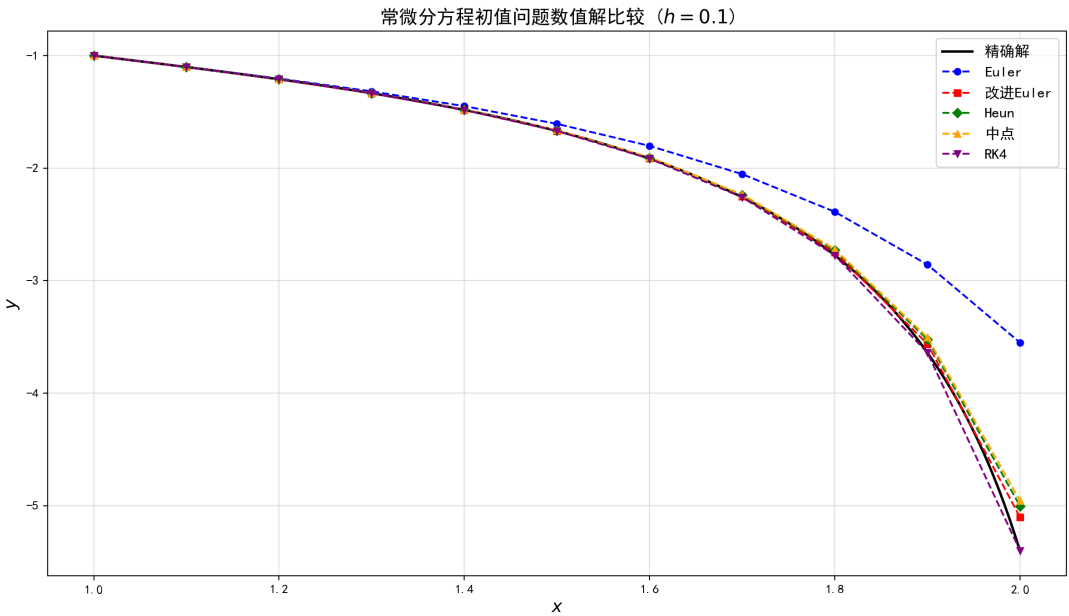


图 1: 不同数值方法与精确解的曲线对比 ($h = 0.1$)

从表和图中可以得出：

- Euler 方法为一阶方法，误差较大（约 1.6×10^{-3} ）。
- 改进 Euler 方法、Heun 方法和中点方法均为二阶方法，误差量级分别为 10^{-5} 、 10^{-6} 和 10^{-6} ，其中 Heun 方法在本问题上表现略优于中点方法。
- 经典四阶 Runge-Kutta 方法精度最高，误差在 10^{-8} 量级，几乎与精确解重合。
- 所有方法均能正确反映解的下降趋势，二阶以上方法已能给出非常满意的结果。

5 附录：Python 程序代码

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import math
4
5 # 设置中文字体
6 plt.rcParams['font.sans-serif'] = ['SimHei', 'DejaVuSans', 'ArialUnicodeMS']
7 plt.rcParams['axes.unicode_minus'] = False
8
9 # 微分方程右端函数
10 def f(x, y):
11     return -1/x**2 - y/x - y**2
12
13 # 精确解
14 def y_exact(x):
15     # y = -tan(ln(x) + pi/4) / x
16     return -np.tan(np.log(x) + np.pi/4) / x
17
18 # ----- 数值方法 -----
19 def euler_step(x, y, h):
20     return y + h * f(x, y)
21
22 def improved_euler_step(x, y, h):
23     y_p = euler_step(x, y, h)
24     return y + h/2 * (f(x, y) + f(x+h, y_p))
25
26 def heun2_step(x, y, h):
27     K1 = f(x, y)
28     K2 = f(x + 2/3*h, y + 2/3*h*K1)
29     return y + h/4 * (K1 + 3*K2)
30
31 def midpoint_step(x, y, h):
32     return y + h * f(x + h/2, y + h/2 * f(x, y))
33
34 def rk4_step(x, y, h):
35     K1 = f(x, y)
36     K2 = f(x + h/2, y + h/2 * K1)
37     K3 = f(x + h/2, y + h/2 * K2)
38     K4 = f(x + h, y + h * K3)
39     return y + h/6 * (K1 + 2*K2 + 2*K3 + K4)
40
41 # ----- 主计算 -----
42 def solve_ivp(step_method, x0, y0, h, N):
43     x = np.zeros(N+1)
44     y = np.zeros(N+1)
45     x[0], y[0] = x0, y0
46     for i in range(N):
47         y[i+1] = step_method(x[i], y[i], h)
48         x[i+1] = x[i] + h
49     return x, y

```

```

50
51 x0, y0 = 1.0, -1.0
52 h = 0.1
53 N = 10
54
55 methods = {
56     "Euler":          euler_step,
57     "改进Euler":      improved_euler_step,
58     "Heun":           heun2_step,
59     "中点":           midpoint_step,
60     "RK4":            rk4_step
61 }
62
63 results = {}
64 for name, method in methods.items():
65     x_vals, y_vals = solve_ivp(method, x0, y0, h, N)
66     results[name] = (x_vals, y_vals)
67
68 # 精确解
69 x_dense = np.linspace(1, 2, 200)
70 y_dense = y_exact(x_dense)
71
72 # ----- 表格输出 -----
73 print("\n节点与精确解", end="")
74 for name in methods:
75     print(f"与{name}数值解与{name}误差", end="")
76 print("\n\\")
77 print("\\hline")
78
79 for i in range(N+1):
80     x_i = 1.0 + i*h
81     y_e = y_exact(x_i)
82     print(f"{x_i:.1f}与{y_e:.6f}", end="")
83     for name in methods:
84         y_n = results[name][1][i]
85         err = abs(y_n - y_e)
86         # 根据误差大小科学计数
87         if err == 0:
88             err_str = "0.0e+00"
89         else:
90             err_str = f"{err:.1e}"
91         # 格式化指数为两位数字
92         print(f"{y_n:.6f}与{err_str}", end="")
93     print("\n\\")
94
95 # ----- 绘图 -----
96 plt.figure(figsize=(12, 7))
97 plt.plot(x_dense, y_dense, 'k-', linewidth=2, label='精确解')
98
99 colors = ['blue', 'red', 'green', 'orange', 'purple']

```

```

100 markers = ['o', 's', 'D', '^', 'v']
101 for (name, (x, y)), c, m in zip(results.items(), colors, markers):
102     plt.plot(x, y, marker=m, color=c, linestyle='--', label=name, markersize=5)
103
104 plt.xlabel('$x$', fontsize=14)
105 plt.ylabel('$y$', fontsize=14)
106 plt.title('常微分方程初值问题数值解比较_($h=0.1$)', fontsize=15)
107 plt.legend(fontsize=12)
108 plt.grid(alpha=0.4)
109 plt.tight_layout()
110 plt.savefig('comparison_plot.png', dpi=200)
111 plt.show()

```